



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

**CS0065
(INTELLIGENT SYSTEMS)**

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Justin D. Vergara
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects
 - If-Else statements
 - Loops (for loop)
 - Functions / methods
2. Understand the environment:
 - Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python38-64\python.exe
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:

- Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
 - The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```

C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\Scripts\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |

```

Your Code - Laboratory Tasks

Task 1, 2, 3 and Bonus Task Source Code:

```
import random

# TASK 1: SET UP THE ENVIRONMENT
class TwoRoomEnvironment:
    """Represents the 2-room environment (Rooms A and B)."""
    def __init__(self, state_a="Clean", state_b="Clean"):
        # Initializing two rooms using a Python dictionary
        self.rooms = {
            'A': state_a.capitalize(),
            'B': state_b.capitalize()
        }

    def is_dirty(self, room: str) -> bool:
        """Function to check if a specified room is dirty."""
        return self.rooms.get(room) == "Dirty"

    def clean_room(self, room: str):
        """Function to clean a specified room."""
        self.rooms[room] = "Clean"

# TASK 2: IMPLEMENT THE RULE-BASED AGENT
class RuleBasedVacuumAgent:
    """Represents the rule-based reflex agent."""
    def __init__(self, initial_location='A'):
        self.current_location = initial_location.upper()

    def move(self):
        """Switches the agent's location between Room A and Room B."""
        self.current_location = 'B' if self.current_location == 'A' else 'A'

    def perceive_and_act(self, env: TwoRoomEnvironment) -> str:
        """Perceives environment state and acts according to predefined rules."""
```

```

curr = self.current_location

# Rule 1: If current room is dirty, clean it.
if env.is_dirty(curr):
    env.clean_room(curr)
    return f"Cleaned Room {curr}"

# Rule 2: If current room is clean, move to the other room.
else:
    self.move()
    return f"Moved to Room {self.current_location}"

# TASK 3: RUN THE SIMULATION (2 ROOMS) WITH RETRY OPTION
def run_task3_simulation():
    while True:
        print("\nTASKS 1, 2, & 3: 2-ROOM AGENT SIMULATION")

        # Task 1: User inputs for environment setup
        state_a = input("Enter initial state for Room A (Clean/Dirty):").strip().capitalize()
        state_b = input("Enter initial state for Room B (Clean/Dirty):").strip().capitalize()
        start_loc = input("Enter agent starting room (A/B):").strip().upper()
        if start_loc not in ['A', 'B']:
            start_loc = 'A'

        env = TwoRoomEnvironment(state_a=state_a, state_b=state_b)
        agent = RuleBasedVacuumAgent(initial_location=start_loc)

        try:
            steps = int(input("Enter number of simulation steps to run:"))
        except ValueError:
            print("Invalid input! Defaulting to 4 steps.")
            steps = 4

```

```

        print("\n" + "-" * 65)
        print(f"INITIAL STATE: {env.rooms} | AGENT START: Room {agent.current_location}")
        print("-" * 65)

    # Task 3: Simulation loop
    for step in range(1, steps + 1):
        loc_before = agent.current_location
        action = agent.perceive_and_act(env)

        print(f"\nStep {step}:")
        print(f"  Agent Location Before Action : Room {loc_before}")
        print(f"  Action Taken                        : {action}")
        print(f"  Current Environment State          : {env.rooms}")

    print("Simulation Complete.")

    # Retry prompt
    retry = input("\nDo you want to retry this 2-Room simulation? (y/n): ").strip().lower()
    if retry != 'y':
        print("Returning to main menu...\n")
        break

# OPTIONAL (BONUS TASK): 3 ROOMS WITH TEXT-GRID VISUALIZATION & RETRY
class ThreeRoomEnvironment:
    """Environment for Rooms A, B, and C."""
    def __init__(self, a, b, c):
        self.rooms = {'A': a.capitalize(), 'B': b.capitalize(), 'C': c.capitalize()}

    def is_dirty(self, room):
        return self.rooms.get(room) == "Dirty"

    def clean(self, room):
        self.rooms[room] = "Clean"

```

```

def get_dirty_rooms(self):
    return [r for r, s in self.rooms.items() if s == "Dirty"]

class BonusAgent:
    """Agent that navigates 3 rooms and targets dirty rooms randomly."""
    def __init__(self, start='A'):
        self.location = start.upper()

    def perceive_and_act(self, env: ThreeRoomEnvironment):
        curr = self.location
        if env.is_dirty(curr):
            env.clean(curr)
            return f"Cleaned Room {curr}"

        dirty_rooms = [r for r in env.get_dirty_rooms() if r != curr]
        if dirty_rooms:
            self.location = random.choice(dirty_rooms)
            return f"Moved to Dirty Room {self.location}"
        else:
            other_rooms = [r for r in ['A', 'B', 'C'] if r != curr]
            self.location = random.choice(other_rooms)
            return f"Patrolled to Clean Room {self.location}"

def render_grid(env_rooms, agent_loc):
    """Evenly spaced 1x3 grid box with aligned boundary borders."""
    row_strings = []
    for r in ['A', 'B', 'C']:
        status = env_rooms[r]
        agent_str = "[*AGENT*]" if agent_loc == r else ""
        content = f"Room {r}: {status} {agent_str}".strip()
        row_strings.append(f"| {content:<24} ")

    border = "+-----" * 3 + "+"
    print(border)
    print("".join(row_strings) + "|")

```

```

print(border)

def run_bonus_simulation():
    while True:
        print("\nBONUS TASK: 3-ROOM GRID SIMULATION")

        state_a = input("Enter state for Room A (Clean/Dirty): ").strip().capitalize()
        state_b = input("Enter state for Room B (Clean/Dirty): ").strip().capitalize()
        state_c = input("Enter state for Room C (Clean/Dirty): ").strip().capitalize()
        start_loc = input("Enter starting room (A/B/C): ").strip().upper()
        if start_loc not in ['A', 'B', 'C']:
            start_loc = 'A'

        try:
            steps = int(input("Enter number of simulation steps: "))
        except ValueError:
            print("Invalid input! Defaulting to 4 steps.")
            steps = 4

        b_env = ThreeRoomEnvironment(state_a, state_b, state_c)
        b_agent = BonusAgent(start_loc)

        print("\nInitial Grid:")
        render_grid(b_env.rooms, b_agent.location)

        for s in range(1, steps + 1):
            act = b_agent.perceive_and_act(b_env)
            print(f"\nStep {s}: {act}")
            render_grid(b_env.rooms, b_agent.location)

        print("\n" + "=" * 65)
        print("Bonus Simulation Complete.")
        print("\n" + "=" * 65)

```

```

        # Retry prompt
        retry = input("\nDo you want to retry this 3-Room simulation?
(y/n): ").strip().lower()
        if retry != 'y':
            print("Returning to main menu...\n")
            break

# MAIN EXECUTION MENU
if __name__ == "__main__":
    while True:
        print("\nCS0065 TECHNICAL ASSESSMENT 2")
        print("1. Run Tasks 1, 2 & 3 (2-Room Simulation)")
        print("2. Run Bonus Task (3-Room Text-Grid Simulation)")
        print("3. Exit")

        choice = input("\nSelect an option (1/2/3): ").strip()

        if choice == '1':
            run_task3_simulation()
        elif choice == '2':
            run_bonus_simulation()
        elif choice == '3':
            print("Exiting program. Goodbye!")
            break
        else:
            print("Invalid selection. Please choose 1, 2, or 3.")

```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. The agent follows two core condition-action rules: if the current room is dirty, it cleans the room, if the current room is clean, it moves to another room (toggling between A and B in the two-room simulation, or targeting dirty/patrol rooms in the three-room simulation).
2. The agent decides its action during each step by checking the cleanliness status of its current location, opting to clean if the state is dirty or executing a move function to change its location if the state is clean.
3. If all rooms start in a clean state, the agent continuously triggers its movement rule and loops back and forth between rooms indefinitely without executing any cleaning action.
4. To handle more than two rooms, the agent's movement logic must be updated from a binary toggle to a system that tracks multiple room keys in a dictionary and navigates through them using cyclic indexing, adjacency graphs, or dynamic lists of dirty rooms.